PyGameMaker — Tutorial 5: Sokoban — Teacher Guide

Companion for the in-app tutorial (**Help > Tutorials > Sokoban: Box-Pushing Puzzle**, 4 pages) and the matching student handout and worksheet. Students should have finished Tutorial 2 or 3 (events, collisions, solid objects).

Overview

Students build a Sokoban puzzle in three phases: a player that moves on a grid, crates that can be pushed, and targets with visual feedback. New ideas: **grid movement** (fixed 32-pixel steps), the **Key press** event, **If can push**, **changing a sprite** with a Step event, and simple **level design**.

> Puzzle design is the real lesson. Ask students to solve each other's levels: a level with fewer targets than crates, or with a crate in a corner, cannot be solved.

Suggested Timing (45-60 minutes)

The tutorial says 20-25 minutes for a confident student; in class allow more.

Segment	Time	What happens
Recap and predict	5 min	Show a Sokoban level; ask what rules the game has
Phase 1: player and walls	10-15 min	Pages 1-2
Phase 2: pushing crates	10-15 min	Page 3
Phase 3: targets and controller	15 min	Page 4; includes the placement-order point below
Swap and solve	5 min	Students play each other's levels
Worksheet / exit ticket	5 min	Worksheet parts A-C

For a 45-minute slot, stop after Phase 2 and finish Phase 3 next time.

Step-by-Step Walkthrough and Common Issues

Phase 1: player and walls

- Use **Key press** events (fire once per press), not *Keyboard (held)*. With held keys the player would run.
- *Move grid* size **32** matches the room grid of 32. The room is 320×320, and grid snap 32×32 in the room editor keeps objects aligned; unaligned objects make the movement look wrong.
- The player needs **Collision with obj_wall** with *Stop movement*, and obj_wall must be Solid.

Phase 2: pushing crates

- The push happens in the **player's** collision event: *If can push (facing)* then *Push other instance 32*. If the push is blocked (wall or another crate behind), the player stops.
- The crate must be Solid and also needs its own **Collision with obj_wall** event with *Stop movement*.
- A crate against a wall (or in a corner) at the start is stuck; tell students not to place crates in corners.

Phase 3: targets and controller

- obj_target must **not** be Solid, or crates and player could never stand on it.
- The crate's Step event checks *If colliding with obj_target* and switches between spr_crate_ok and spr_crate.
- **Draw order is placement order.** The room draws instances in the order they were placed, and there is no Depth setting or "bring to front" in the room editor. A target placed after the player and crates is drawn on top of them: the crate does turn green, but it is hidden under the red marker, and the player seems to vanish on a target. The tutorial now tells students to place targets first, then delete and re-place the player and crates. Watch for this: it looks like "the green crate feedback does not work".
- The instruction text uses the default draw colour, which is black; it sits over the top wall row of the room, so it is readable. Use *Set draw color* if you move it over a black area.
- The tutorial has no win message; students see success when every crate is green. A win check ("all crates on targets") is a good challenge.

> **Reference projects.** Download the checkpoint projects from the wiki (05_sokoban_checkpoints.zip): one project for the end of each phase, with the tutorial's example level. Give a stuck student the previous phase.

Vocabulary Introduced

Term	What it means here
Grid movement	Moving a fixed distance (32 pixels) per key press
Key press	An event that fires once when a key goes down
If can push	A check that the space behind the crate is free
Sprite swap	Changing an object's picture with *Set sprite*
Level design	Deciding where walls, crates and targets go so the puzzle can be solved

Discussion Questions

- Why one event per key press, not per frame?
- What makes a Sokoban level impossible? (A crate in a corner, too few targets.)
- How could the game know that the puzzle is solved?
- What changes if the targets are Solid? Try it.

Differentiation

- **Support:** give the Phase 2 checkpoint; use the tutorial's example level so everyone has a solvable puzzle.
- **Extension:** move counter; several rooms with harder levels; an undo key; a "level complete" message.

Worksheet Answer Key

Part A: 1-C, 2-A, 3-D, 4-B.

Part B: 1. obj_player. 2. obj_crate. 3. obj_crate (its Step event). 4. obj_controller. 5. obj_wall (and Solid crates block the player).

Part C:

1. Crates and the player could not enter a Solid object's square, so nothing could ever reach a target.
2. The player does not move and the crate stays: *If can push* finds the space behind the crate blocked, so it runs *Stop movement* instead.
3. No, a level needs at least as many targets as crates. Always count them and solve your own level before sharing it.

Parts D and E: completion and reflection.

Rubric: Sokoban

Level	What the game shows
4 - Complete	Grid movement; walls stop player and crates; crates push only when the space behind is free; crates turn green on targets; restart with R; a solvable level with equal crates and targets
3 - Working	Movement, walls and pushing work; feedback or restart missing
2 - Partly there	Player moves on the grid, but crates cannot be pushed
1 - Started	Player and walls exist; movement is not one square per press

End-of-Session Checklist

- [ ] Every student has a saved project (Ctrl+S)
- [ ] Every student reached at least Phase 2 (a crate can be pushed)
- [ ] Note who needs the checkpoint project next time